1. A. **PLA:** PLA stands for Programmable Logic Array. PLA is a kind of programmable logic device used to implement combinational logic circuits. PLA is a fixed architecture logic device with programmable AND gates followed by programmable OR gates which can then be conditionally complemented to produce an output. PLA is a type of programmable logic device used to build a reconfigurable digital circuit. PLA is a combination of memory and logic. It has 2n AND gates for N input variables, and for M outputs from PLA, there should be M OR gates, each with programmable inputs from all of the AND gates. The logic diagram for PLA is followed below.

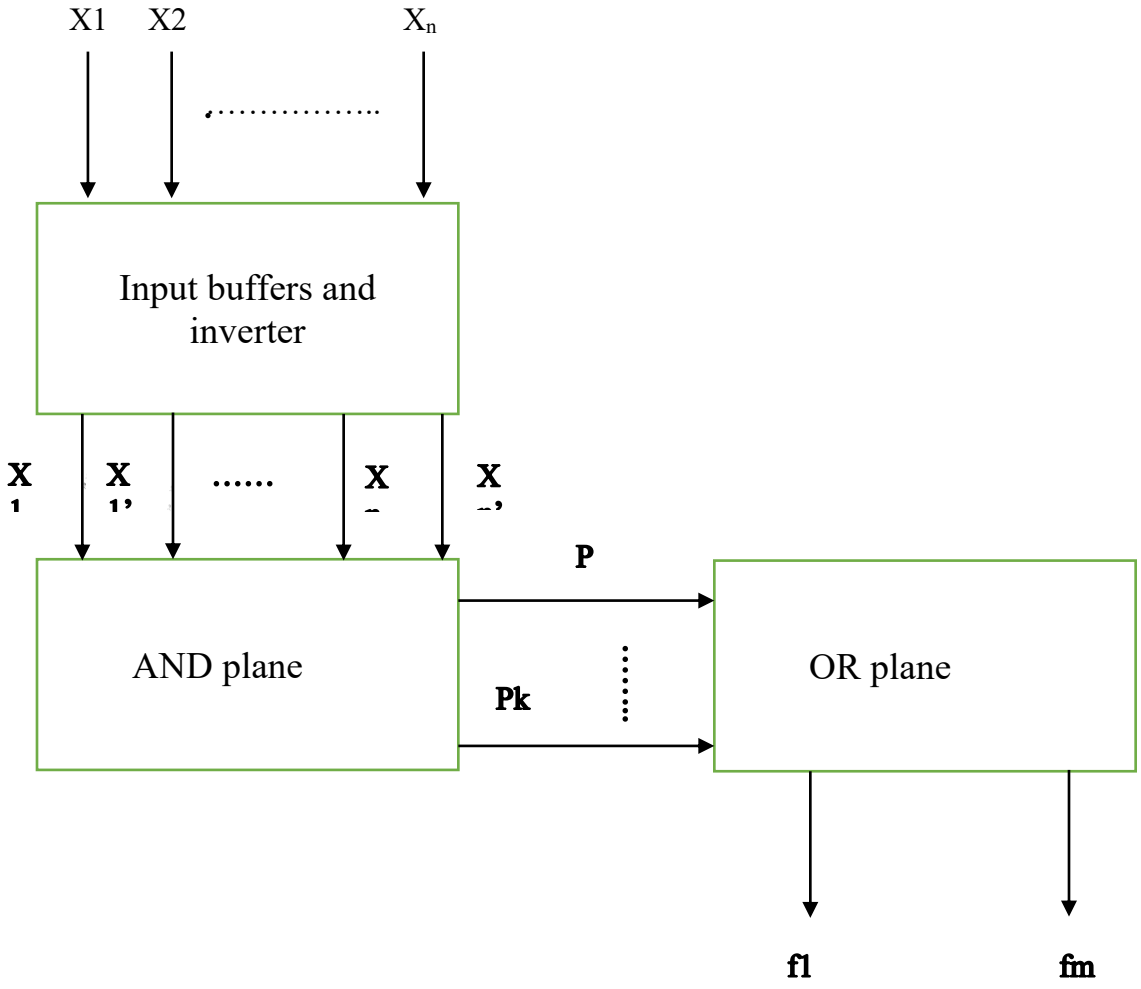


Fig: General structure of PLA
(source: Slides from lecture in moodle)

B. **PAL:** PAL stands for programmable array logic. PAL is a family of programmable logic device semiconductors used to implement logic functions in digital circuits introduced by Monolithic Memories Inc.(MMI) in march 1978. MMI also registered a trademark on the word PAL for use in programmable semiconductor logic circuits. PAL devices consisted of a small PROM (programmable read-only memory) core and additional output logic used to implement particular desired logic functions with few components. It has a programmable AND array and fixed OR array. Because only the AND array is programmable, it is easier to use but not flexible as compared to Programmable Logic Array (PLA). PAL's only limitation is the number of AND gates. PAL consists of small programmable read-only memory (PROM) and additional output logic used to implement a particular desired logic function with limited components.

C. **CPLD**: A complex programmable logic device (CPLD) is a programmable logic device with complexity between that of PALs and FPGAs and architectural features of both. The main building block of the CPLD is a macrocell, which contains logic implementing disjunctive normal form expressions and more specialized logic operations. The AND/OR array is reprogrammable and can perform a multitude of logic functions. Macrocells are functional blocks that perform combinatorial or sequential logic and also have the added flexibility for true or complement, along with varied feedback paths. Traditionally, CPLDs have used analog sense amplifiers to boost the performance of their architectures. This performance boost came at the cost of very high current requirements. CoolRunner™-II CPLDs, created by Xilinx, use an innovative all-digital core to achieve the same performance levels at ultra-low power requirements. This allows designers to use the same CPLD architecture for high-performance and low-power designs.

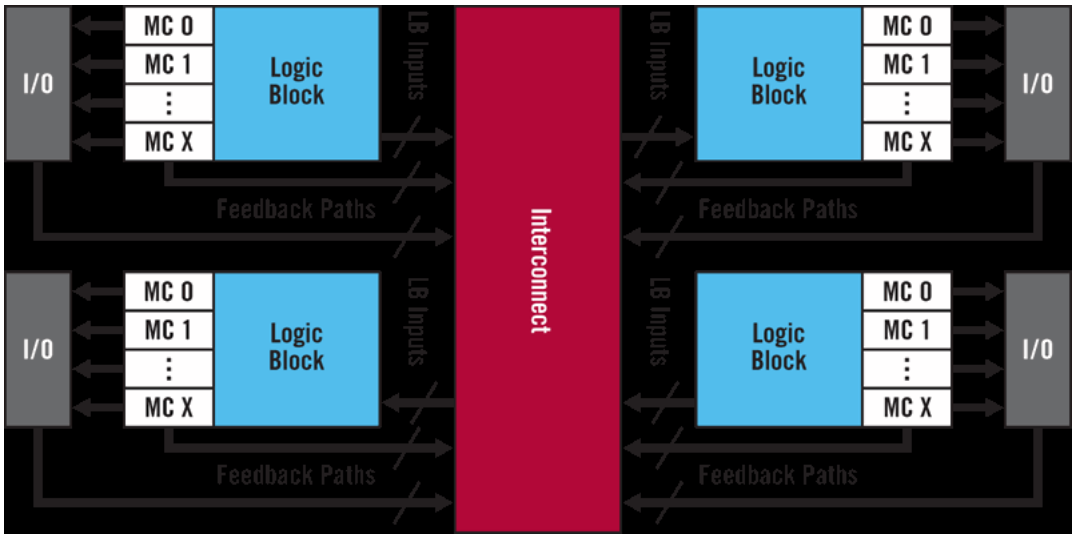


Fig: CoolRunner™-II CPLDs
Source: https://www.xilinx.com/products/silicon-devices/cpld/cpld.html

D. **LUT:** A lookup table (LUT) is a fast way to realize a complex function in digital logic. The address is the function input, and the value at that address is the function output. The advantage is that computing the function only takes a single memory lookup regardless of the complexity of the function, so it is very fast. LUT usually has two or more outputs,not just one output.

The disadvantage is that it takes memory, especially if someone needs high resolution for the function input. For example, the sine function is often implemented as a table lookup. If 10-bit angles are good enough answer, then the whole function can be implemented as a lookup table with 1024 entries. (Actually, in the case of sine, only 1/4 cycle is stored, then negated or indexed backward depending on the actual quadrant, but that is an aside specific to sine).

The function input can also be a combination of different input variables with the result expressed as a single integer. For example, a 4 x 4 bit multiply can be implemented as a lookup table of 256 values. The 8-bit table address can be the two 4-bit input values concatenated.

E. **FPGA:** FPGA stands for field programmable gate array. Field Programmable Gate Arrays (FPGAs) are semiconductor devices based around a matrix of configurable logic blocks (CLBs) connected via programmable interconnects. FPGAs can be reprogrammed to desired application or functionality requirements after manufacturing. It is a semiconductor IC where a large majority of the electrical functionality inside the device can be changed, changed by the design engineer, altered during the PCB assembly process, or even changed after the equipment has been shipped to customers out in the 'field.' FPGAs benefit designers of many types of electronic equipment, ranging from smart energy grids, aircraft navigation, automotive driver's assistance, medical ultrasounds, data center search engines, etc. FPGA functionality can change upon every power-up of the device. So, when a design engineer wants to make a change, they can download a new configuration file into the machine and try out the change, which brings flexibility. While ASICs may cost less per unit than an equivalent FPGA, building them requires a non-recurring expense (NRE), expensive software tools, specialized design teams, and long manufacturing cycles. For example, Intel FPGAs support long lifecycles (15 years or more), avoiding the cost of redesigning and requalifying OEM production equipment if one of the electronic devices on board goes end of life (EOL). So, FPGA is low in cost than ASIC.

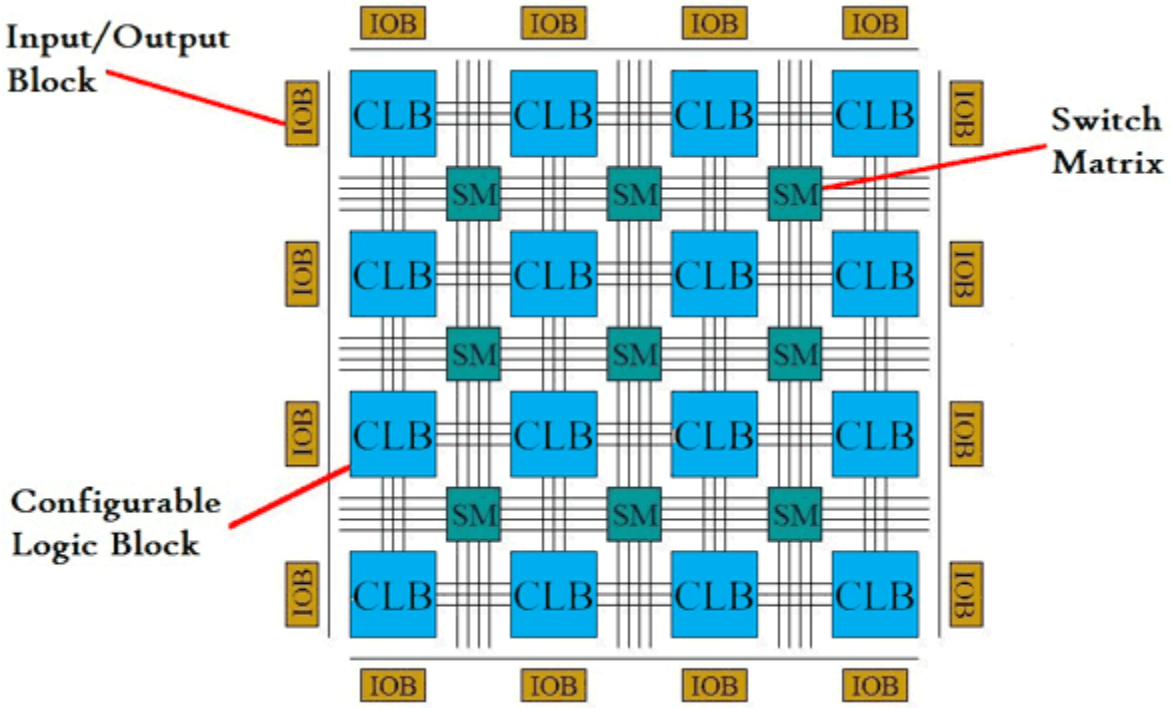


Fig: FPGA structure

Source: https://circuitdigest.com/tutorial/what-is-fpga-introduction-and-programming-tools

F. **Cyclone V:** Cyclone® V FPGAs have lower total power than the previous generation, efficient logic integration capabilities, integrated transceiver variants, and SoC FPGA variants with an ARM*-based hard processor system (HPS). The product family is recommended for Intel Edge-Centric applications and designs. Cyclone® V FPGAs provide the industry's lowest system cost and power, along with performance levels that make the device family ideal for differentiating your high-volume applications. It reduces almost 40% cost of the previous generation. To date, it has released six variants of models. With Cyclone® V FPGAs, someone can get the power, cost, and performance levels that need for high-volume applications, including protocol bridging, motor control drives, broadcast video converters, capture cards, and handheld devices. The FPGA logic fabric lets differentiate your system by implementing custom IP or off-the-shelf preconfigured IP from Intel or its partners into your designs which allows adapting quickly to varying or changing protocols, standards, and interfaces. It preserves the flexibility of processor boot or FPGA configuration sequence, system response to processor reset, and independent memory interfaces of a two-chip solution. In addition, it maintains data integrity and reliability with integrated ECC and protects DRAM memory shared by the processor and FPGA with an integrated memory protection unit.

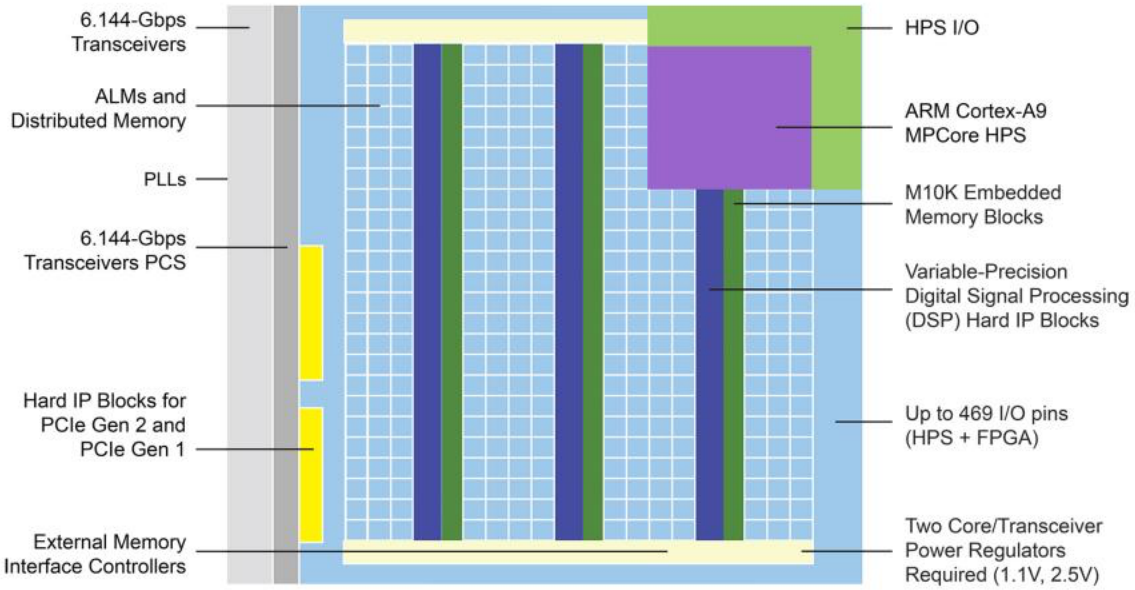


Fig: Architecture of cyclone V

Source: https://www.intel.com/content/www/us/en/products/details/fpga/cyclone/v.html

G. **DE1 SOC BOARD:** The DE1-SoC Development Kit presents a robust hardware design platform built around the Altera System-on-Chip (SoC) FPGA, which combines the latest dual-core Cortex-A9 embedded cores with industry-leading programmable logic for ultimate design flexibility. Users can now leverage the power of tremendous re-configurability paired with a high-performance, low-power processor system. Altera's SoC integrates an ARM-based hard processor system (HPS) consisting of processor, peripherals and memory interfaces tied seamlessly with the FPGA fabric using a high-bandwidth interconnect backbone. The DE1-SoC development board includes hardware such as high-speed DDR3 memory, video and audio capabilities, Ethernet networking, and much more. The DE1-SoC Development Kit contains all components needed to use the board in conjunction with a computer that runs the Microsoft Windows XP or later.

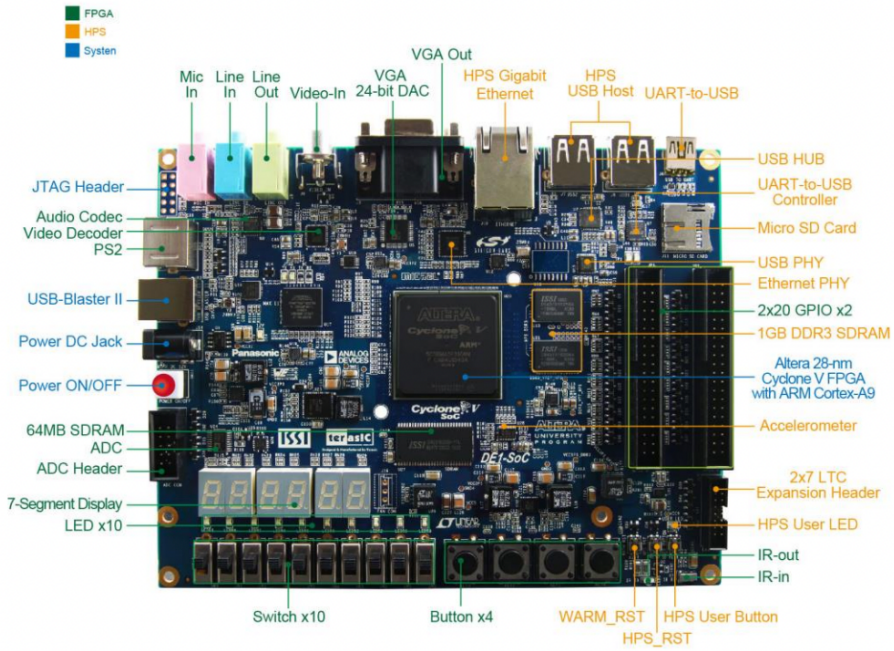


Fig: Development Board (DE1-soc)

Source: http://www.ee.ic.ac.uk/pcheung/teaching/ee2_digital/DE1-SoC_User_manual.pdf

The below diagram gives the block diagram of the board. To provide maximum flexibility for the user, all connections are made through the Cyclone V SoC FPGA device. Thus, the user can configure the FPGA to implement any system design.

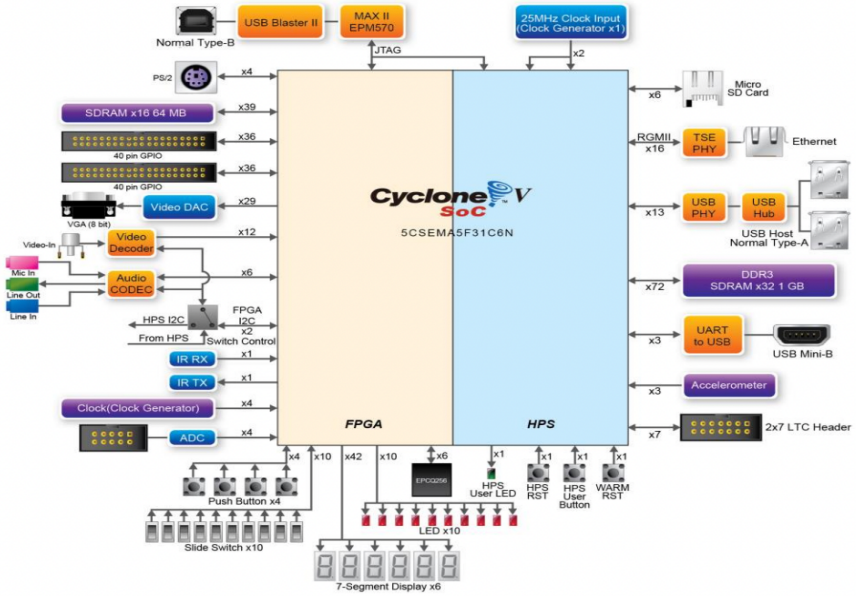


Fig: Development Board (DE1-soc)

Source: http://www.ee.ic.ac.uk/pcheung/teaching/ee2_digital/DE1-SoC_User_manual.pdf

The DE1-SoC board has many features that allow users to implement a wide range of designed circuits, from simple circuits to various multimedia projects.
The following hardware is provided on the board:

- FPGA
    - Altera Cyclone® V SE 5CSEMA5F31C6N device
    - Altera Serial Configuration device – EPCQ256
    - USB Blaster II(on board) for programming; JTAG Mode
    - 64MB SDRAM (16-bit data bus)
    - 4 Push-buttons
    - 10 Slide switches
    - 10 Red user LEDs
    - Six 7-segment displays
    - Four 50MHz clock sources from clock generator
    - 24-bit CD-quality audio CODEC with line-in, line-out, and microphone-in jacks
    - VGA DAC (8-bit high-speed triple DACs) with VGA-out connector
    - TV Decoder (NTSC/PAL/SECAM) and TV-in connector
    - PS/2 mouse/keyboard connector
    - IR receiver and IR emitter
    - Two 40-pin Expansion Header with diode protection
    - A/D Converter, 4-pin SPI interface with FPGA
- HPS (Hard Processor System)
    - 800MHz Dual-core ARM Cortex-A9 MPCore processor
    - 1GB DDR3 SDRAM (32-bit data bus)
    - 1 Gigabit Ethernet PHY with RJ45 connector
    - 2-port USB Host, Normal Type-A USB connector
    - Micro SD card socket
    - Accelerometer (I2C interface + interrupt)
    - UART to USB, USB Mini-B connector
    - Warm reset button and cold reset button
    - One user button and one user LED
    - LTC 2x7 expansion header

2. The list of boolean Algebra postulates and theorems are below:

Postulates of the Boolean algebra:

#	**Name of the Postulates**	**Postulate Equation**
1	Law of Identity	$A + 0 = 0 + A = A$ $A \cdot 1 = 1 \cdot A = A$

#	Name of the Postulates	Postulate Equation
2	Commutative Law	(A + B) = (B + A) (A . B) = (B . A)
3	Distributive Law	A . (B + C) = (A . B) + (A . C) A + (B . C) = (A + B) . (A + C)
4	Associative Law	A + (B + C) = (A + B) + C (A . B) . C = A . (B . C)
5	Complement Law	A + A' = 1 A . A' = 0

Theorem of Boolean algebra:

#	Theorem	Statement	Equations
1	Duality Theorem	A boolean relation can be derived from another boolean relation by changing OR sign to AND sign and vice versa and complementing the 0s and 1s.	A + A' = 1 and A . A' = 0 are the dual relations.
2	DeMorgan's Theorem 1	Complement of a product is equal to the sum of its complement.	(A . B)' = A' + B'
3	DeMorgan's Theorem 2	Complement of a sum is equal to the product of the complement.	(A + B)' = A' . B'
4	Idempotency Theorem	–	A + A = A A . A = A
5	Involution Theorem	–	A" = A
6	Absorption Theorem	–	A + (A . B) = A A . (A + B) = A
7	Associative Theorem	–	A + (B + C) = (A + B) + C A . (B . C) = (A . B) . C

#	Theorem	Statement	Equations
8	Consensus Theorem	–	AB + A'C + BC = AB + A'C (A + B) + (A' + C) + (B + C) = (A + B) + (A' + C)
9	Uniting Theorem	–	AB + AB' = A (A+B) + (A + B') = A
10	Other theorems	–	A + 1 = 1 A . 0 = 0
11	Other theorems	–	A + (A' . B) = A + B A . (A' + B) = A . B

3. **Flip flop Excitation Table:** The excitation table has the minimum inputs, which will excite or trigger the flip flop to go from its present state to the next state. It is derived from the truth table. Generally, the operation of each flip flop is explained with the help of the truth table. The truth table has all the input combinations, for which the flip flop reacts to produce the next state output. The excitation table consists of two columns for the present state(Qn) and next state(Qn+1) and one or two columns for each input. The input columns depend on the type of the flip flop.

4. **Maxterm**: A sum that is a logical OR of a set of variables where each individual variable only appears once in the sum, either in complemented or uncomplemented form, so that the value of the sum becomes 0. All the maxterms in a product of maxterms should have the same variables, although each maxterm should differ from every other one by the pattern of complementation of those variables. Each maxterm has a value of 0 for exactly one combination of values of variables. E. g. A + B + C' (001) => M1 (the value is 0). Therefore Mi = m'i . A function can be written as a product of maxterms, which is referred to as a maxterm expansion or a standard product of sums.

   f = (A+B+C)(A+B+C')(A+B'+C)
   f (A,B,C) = M0M1M2 or
   f (A,B,C) = Π M (0,1,2)

   Minterm: A minterm of n variables = product of n literals in which each variable appears exactly once either in T or F form, but not in both. (Also known as a standard product term). Each minterm has value 1 for exactly one combination of values of variables. E.g. ABC (111) => m7. A function can be written as a sum of minterms, which is referred to as

a minterm expansion or a standard sum of products. Minterms present in f correspond with the 1's of f in the truth table.

Sum of product form: Sum of Product is the abbreviated form of SOP. Sum of product form is a form of expression in Boolean algebra in which different product terms of inputs are being summed together. This product is not arithmetical multiply but it is Boolean logical AND and the Sum is Boolean logical OR. There are few different forms of Sum of Product.

- Canonical SOP Form
- Non-Canonical SOP Form
- Minimal SOP Form

Product of sum form: The product of Sum form is a form in which products of different sum terms of inputs are taken. These are not arithmetic product and sum but they are logical Boolean AND and OR respectively. For more presious about pos , someone need to know the maxterm. The product of the sum is classified into three types which include the following.

- Canonical Product of Sums
- Non – Canonical Product of Sums
- Minimal Product of Sums